Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Уральский федеральный университет
имени первого Президента России Б.Н. Ельцина»

Институт радиоэлектроники и информационных технологий-РТФ

ОТЧЕТ

о выполнении итогового домашнего задания
по дисциплине «Программная инженерия»

на тему «Мониторинг доступности API, анализ логов
и контейнеризация сервиса»

Выполнил:
студент группы РИМ-160980к
Трофимов Н.А.

Проверил:
Михаил Меркушов

Екатеринбург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 Организация проекта и работа с Git 4
1.1 Репозиторий и ветки 4
1.2 Файлы .gitignore и .gitattributes 4
1.3 Коммиты и Pull Request 5
2 Анализ сетевого трафика 6
2.1 Захват трафика 6
2.2 Результаты анализа 6
3 Bash-скрипты для мониторинга и анализа 9
3.1 Скрипт мониторинга API 9
3.2 Скрипт анализа логов 10
3.3 Проверка скриптов 13
4 Docker Compose для стека мониторинга 14
5 Конвейер CI/CD в GitHub Actions 16
6 Передача файлов на удаленный сервер по SSH и SCP 18
ЗАКЛЮЧЕНИЕ 20
ПРИЛОЖЕНИЕ А. Листинги скриптов и конфигурации 21

ВВЕДЕНИЕ

Цель работы: закрепить навыки написания Bash-скриптов, анализа сетевого трафика, работы с Git, настройки CI/CD и контейнеризации готовых сервисов.

По легенде задания команда стартапа написала простой API-сервис (echo-сервер). DevOps-инженеру нужно настроить мониторинг его доступности, автоматизировать разбор логов и подготовить окружение для развертывания сервиса в контейнерах. Работа разбита на шесть этапов:

1) организация репозитория на GitHub: защищенная ветка main, рабочая ветка feature/monitoring и Pull Request;

2) захват и анализ в Wireshark HTTP-трафика к публичному API;

3) Bash-скрипты для проверки доступности API и анализа журнала веб-сервера;

4) стек из echo-сервиса и обратного прокси nginx в Docker Compose;

5) конвейер CI/CD в GitHub Actions;

6) описание передачи проекта на удаленный сервер по SSH и SCP.

Работа выполнялась на компьютере с Windows 11. Скрипты и контейнеры запускались в WSL 2 (Ubuntu 24.04) с Docker Desktop. Исходный код опубликован в репозитории https://github.com/trofim20/infra-monitoring-project, дамп сетевого трафика traffic.pcapng приложен к отчету.

1 Организация проекта и работа с Git

1.1 Репозиторий и ветки

На GitHub создан публичный репозиторий infra-monitoring-project. Первый коммит в ветке main содержит заготовку проекта: README.md, .gitignore и .gitattributes. От этого коммита создана ветка feature/monitoring, и весь остальной код писался в ней.

Для ветки main в настройках репозитория (Settings, Branches) добавлено правило защиты. Изменения попадают в main только через Pull Request, а перед слиянием должны пройти проверки CI lint, test-scripts и test-compose. Прямой push и force push в main запрещены, удалить ветку тоже нельзя.

Файлы проекта перечислены в таблице 1.

Таблица 1 – Файлы проекта

Файл	Назначение
check_api.sh	проверка доступности API по коду HTTP-ответа
analyze_logs.sh	генерация тестового access.log и статистика по нему
docker-compose.yml	стек из сервисов api и proxy
nginx/templates/ default.conf.template	шаблон конфигурации обратного прокси nginx
.github/workflows/ci.yml	конвейер CI в GitHub Actions
README.md	описание проекта, инструкции по запуску, CI и деплою по SSH
.gitignore	файлы, которые не попадают в репозиторий
.gitattributes	окончания строк LF для текстовых файлов

1.2 Файлы .gitignore и .gitattributes

В .gitignore перечислено то, что не должно попадать в репозиторий: журналы (*.log и каталог logs/), в том числе access.log, который analyze_logs.sh создает при каждом запуске; временные файлы редакторов и операционных систем (*.tmp, *.bak, *.swp, *~, .DS_Store, Thumbs.db);

артефакты сборки и архивы (build/, dist/, out/, *.tar.gz, *.zip); файл .env с переменными окружения и настройки IDE (.idea/, .vscode/).

Файл .gitattributes понадобился потому, что проект редактировался в Windows. Там Git может сохранять файлы с окончаниями строк CRLF, а bash в Linux на таком скрипте падает с ошибкой вида $'\r': command not found. Правило * text=auto eol=lf заставляет Git хранить и выгружать все текстовые файлы с окончаниями LF.

1.3 Коммиты и Pull Request

Каждый этап работы оформлен отдельным коммитом, сообщения коммитов написаны на английском языке (таблица 2).

Таблица 2 – Коммиты в репозитории

Коммит	Сообщение	Что сделано
49a54fd	Initial commit: add README, .gitignore and .gitattributes	заготовка проекта в ветке main
bd07b7d	Add check_api.sh to check API availability	скрипт мониторинга API
63c2098	Add analyze_logs.sh to generate and analyze access.log	скрипт анализа логов
cf8f1f9	Add docker-compose stack with http-echo behind nginx proxy	стек api и proxy, шаблон nginx
d045543	Add CI workflow with syntax check and script tests	конвейер CI: задачи lint и test-scripts
2ca3318	Add docker-compose job to CI	задача test-compose в CI
321d365	Describe usage, CI and SSH deployment in README	документация, раздел про деплой по SSH
3ec034d	Merge pull request #1 from trofim20/feature/monitoring	слияние Pull Request #1 в main

Первый коммит сделан в ветке main, следующие шесть в feature/monitoring. Когда работа была готова, открыт Pull Request #1 из feature/monitoring в main (https://github.com/trofim20/infra-monitoring-project/pull/1). В его описании перечислены изменения по каждому этапу, команды для проверки и список задач CI. После того как все проверки на Pull Request прошли успешно (раздел 5), он был слит в main, последний коммит в таблице 2 и есть этот merge-коммит.

2 Анализ сетевого трафика

2.1 Захват трафика

На рабочий компьютер установлен Wireshark 4.6.9 вместе с драйвером захвата пакетов Npcap. Компьютер подключен к сети по кабелю, поэтому захват запускался на проводном интерфейсе Ethernet.

Чтобы в дамп не попали соединения браузера и фоновых программ, перед запуском задан фильтр захвата host httpbin.org. Wireshark определяет адреса этого имени при старте захвата и записывает только пакеты, где они встречаются. Во время захвата в PowerShell выполнены запросы из листинга 1. Первые четыре отправлены по HTTP без шифрования, последний по HTTPS, чтобы сравнить, что видно в дампе в этом случае.

Листинг 1 – Запросы к httpbin.org во время захвата

```
curl.exe "http://httpbin.org/get?source=wireshark"
curl.exe -X POST http://httpbin.org/post -d "service=echo&status=active"
curl.exe -X DELETE http://httpbin.org/delete
curl.exe http://httpbin.org/status/404
curl.exe https://httpbin.org/get
```

Захват длился около 35 секунд, за это время записано 57 пакетов. Дамп сохранен в файл traffic.pcapng, он приложен к отчету. Для просмотра HTTP-трафика применен фильтр отображения http (рисунок 1).

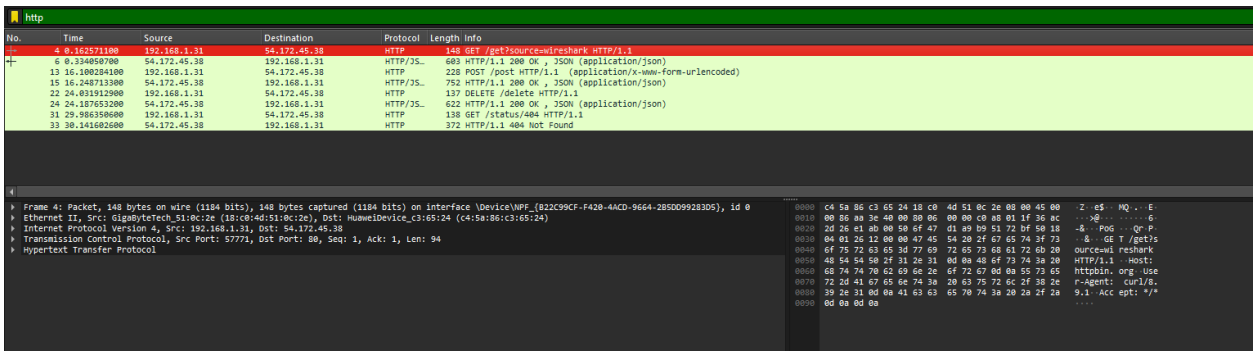


Рисунок 1 – HTTP-трафик в Wireshark после фильтрации

2.2 Результаты анализа

HTTP-запросы и ответы, найденные в дампе, приведены в таблице 3.

Таблица 3 – HTTP-обмен в дампе traffic.pcapng

Кадры	Клиент	Сервер	Запрос	Ответ
4, 6	192.168.1.31:57771	54.172.45.38:80	GET /get?source= wireshark	200 OK
13, 15	192.168.1.31:54385	54.172.45.38:80	POST /post	200 OK
22, 24	192.168.1.31:54429	54.172.45.38:80	DELETE /delete	200 OK
31, 33	192.168.1.31:51792	54.172.45.38:80	GET /status/404	404 Not Found
37-57	192.168.1.31:51798	54.172.45.38:443	зашифрован	зашифрован

IP-адреса. В обмене участвовали два адреса: 192.168.1.31 принадлежит рабочему компьютеру, 54.172.45.38 серверу httpbin.org. Адрес 192.168.1.31 частный, из домашней сети: в интернет компьютер выходит через роутер, который подменяет его своим внешним адресом (NAT). Поэтому в заголовке Ethernet получателем всех исходящих кадров указан роутер, а не сервер. По данным nslookup, имя httpbin.org разрешается в три IPv4-адреса в облаке Amazon AWS (54.172.45.38, 3.221.253.123 и 44.209.237.184), и во всех пяти запросах curl подключался к 54.172.45.38. Каждый вызов curl открывал отдельное TCP-соединение: всего их пять, четыре на порт 80 и одно на порт 443, поэтому порт клиента в каждом запросе свой.

HTTP-методы. В дампе есть запросы GET (/get с параметром source=wireshark и /status/404), POST (/post, в теле переданы данные формы service=echo&status=active с типом application/x-www-form-urlencoded) и DELETE (/delete).

Коды ответов. На запросы GET /get, POST /post и DELETE /delete сервер ответил кодом 200 OK и вернул JSON с параметрами полученного запроса. На GET /status/404 пришел ответ 404 Not Found с пустым телом (Content-Length: 0).

Запрос по HTTPS в фильтр http не попал. Его соединение на порт 443 занимает кадры с 37 по 57, и в них видны только TCP-соединение и TLS-рукопожатие. В сообщении Client Hello (кадр 40) открытым текстом передается имя сервера httpbin.org в расширении SNI. Сервер выбрал TLS 1.3, после рукопожатия идут только записи Application Data, поэтому метод,

путь и код ответа зашифрованы. Найти это соединение можно фильтром tls.handshake.extensions_server_name == "httpbin.org".

3 Bash-скрипты для мониторинга и анализа

3.1 Скрипт мониторинга API

Скрипт check_api.sh (листинг А.1) принимает адрес проверяемого API аргументом командной строки. Ключи разбираются встроенной командой getopts: -t задает таймаут запроса в секундах (по умолчанию 10), -v дополнительно выводит URL и время ответа, -h печатает справку. После ключей должен остаться ровно один аргумент, иначе скрипт выводит ошибку и справку в поток ошибок и завершается с кодом 1.

Перед запросом URL проверяется регулярным выражением, которое собрано из четырех частей: схема http или https, имя хоста из латинских букв, цифр, точек и дефисов, необязательный порт и путь. Значение таймаута тоже проверяется регулярным выражением ^[1-9][0-9]*$.

Сам запрос выполняет curl. Тело ответа отбрасывается (--output /dev/null), а через --write-out скрипт получает код ответа и время запроса. Если код равен 200, выводится «API доступен» и возвращается 0. При любом другом коде выводится «API недоступен, код ответа: XXX» и возвращается 1. Если сервер не ответил совсем, curl подставляет код 000, а скрипт по коду завершения curl пишет в поток ошибок причину: не найден хост, не удалось подключиться, истек таймаут или ошибка TLS.

Результаты проверки скрипта на разных адресах приведены в таблице 4, работа с валидным и невалидным URL показана на рисунке 2. После каждого запуска командой echo $? выведен код возврата.

Таблица 4 – Проверка check_api.sh

Аргумент	Вывод скрипта	Код возврата
https://httpbin.org/status/200	API доступен	0
https://httpbin.org/status/404	API недоступен, код ответа: 404	1
https://httpbin.org/status/503	API недоступен, код ответа: 503	1
http://127.0.0.1:9	API недоступен, код ответа: 000 Причина: не удалось подключиться к серверу	1
not-a-url	Ошибка: некорректный URL	1
без аргументов	Ошибка: нужно передать один аргумент: URL API	1

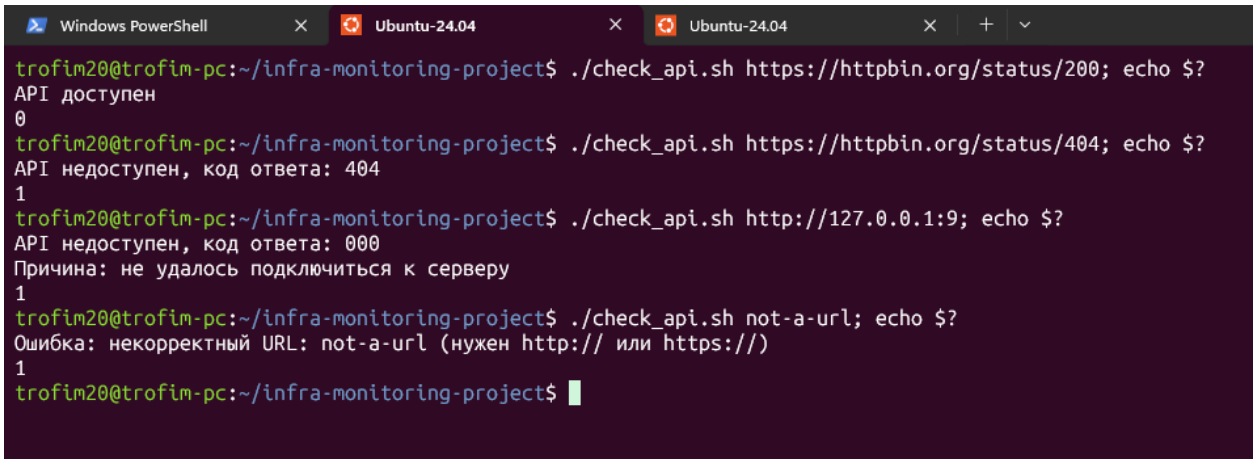


Рисунок 2 – Работа check_api.sh с валидным и невалидным URL

3.2 Скрипт анализа логов

Скрипт analyze_logs.sh (листинг А.2) сначала генерирует тестовый журнал access.log, а затем считает по нему статистику. Строки журнала соответствуют формату из задания (листинг 2).

Листинг 2 – Первые строки сгенерированного access.log

```
192.168.1.33 - - [15/Aug/2026:10:15:32] "GET /api/users HTTP/1.1" 404
192.168.1.25 - - [15/Aug/2026:10:15:34] "GET /api/users HTTP/1.1" 200
10.0.0.5 - - [15/Aug/2026:10:15:35] "GET /api/users HTTP/1.1" 200
203.0.113.45 - - [15/Aug/2026:10:15:35] "DELETE /api/orders/140 HTTP/1.1" 204
```

Для генерации в скрипте заданы три списка: IP-адреса клиентов, запросы и коды ответов. У каждого значения указан вес, и чем он больше,

тем чаще значение встречается в журнале. Функция make_pool разворачивает список в массив, где значение повторяется столько раз, каков его вес, и случайный элемент берется из этого массива. Чтобы журнал был похож на настоящий, в пути вида /api/users/{id} подставляется случайный номер, запрос POST /api/login примерно в трети случаев получает код 401, а успешные POST и DELETE отвечают кодами 201 и 204. Время каждой следующей записи сдвигается на случайное число секунд от 0 до 3.

Случайные числа выдает линейный конгруэнтный генератор, написанный прямо в скрипте. Встроенная переменная RANDOM не подошла, потому что ее алгоритм может отличаться в разных версиях bash. Свой генератор с фиксированным начальным значением дает одинаковый журнал на любой машине, это проверено в Ubuntu 24.04 и в Git Bash в Windows: файлы совпали полностью.

По умолчанию генерируется 500 строк. Ключи скрипта приведены в таблице 5, они разбираются через getopts. Неизвестный ключ, лишний аргумент, нечисловое значение или число строк меньше 300 приводят к ошибке с кодом 1.

Таблица 5 – Ключи analyze_logs.sh

Ключ	Назначение	По умолчанию
-n	число строк в журнале, не меньше 300	500
-f	путь к файлу журнала	access.log
-s	начальное значение генератора случайных чисел	2026
-a	только анализ существующего файла, без генерации	выключен
-h	вывод справки	

Статистика собирается конвейерами из стандартных утилит:

1) общее количество запросов считает grep -cE: учитываются строки, которые целиком подходят под регулярное выражение строки журнала. Строки в другом формате не считаются, и скрипт сообщает, сколько их пропущено;

2) для топ-3 IP-адресов grep -oE извлекает адрес из начала строки, awk отбрасывает адреса с октетом больше 255, конвейер sort | uniq -c | sort -k1,1nr подсчитывает и упорядочивает адреса, а awk печатает первые три;

3) запросы с кодами 4xx и 5xx считает awk, сравнивая последнее поле строки с выражениями ^4[0-9][0-9]$ и ^5[0-9][0-9]$. Второй вызов awk переводит результат в проценты;

4) дополнительно выводятся распределение по всем кодам ответа и три самых популярных эндпоинта. Для эндпоинтов sed вырезает путь из строки запроса и заменяет числовой идентификатор на {id}, чтобы /api/users/7 и /api/users/42 считались одним адресом.

Регулярные выражения, по которым разбирается строка журнала, приведены в листинге 3. Выражение для строки собирается из выражений для IP-адреса, даты и запроса.

Листинг 3 – Регулярные выражения в analyze_logs.sh

```
readonly IP_REGEX='([0-9]{1,3}\.){3}[0-9]{1,3}'
readonly DATE_REGEX='\[[^]]+\]'
readonly REQ_REGEX='"[A-Z]+ [^ "]+ HTTP/[0-9.]+"'
readonly LINE_REGEX="^${IP_REGEX} - - ${DATE_REGEX} ${REQ_REGEX} [0-9]{3}\$"
```

Результат запуска с параметрами по умолчанию показан на рисунке 3. В журнале 500 запросов, больше всего их пришло с адресов 192.168.1.25 (139), 10.0.0.5 (95) и 172.16.5.4 (88). Ответов с кодами 4xx 118 (23,6 %), с кодами 5xx 44 (8,8 %).

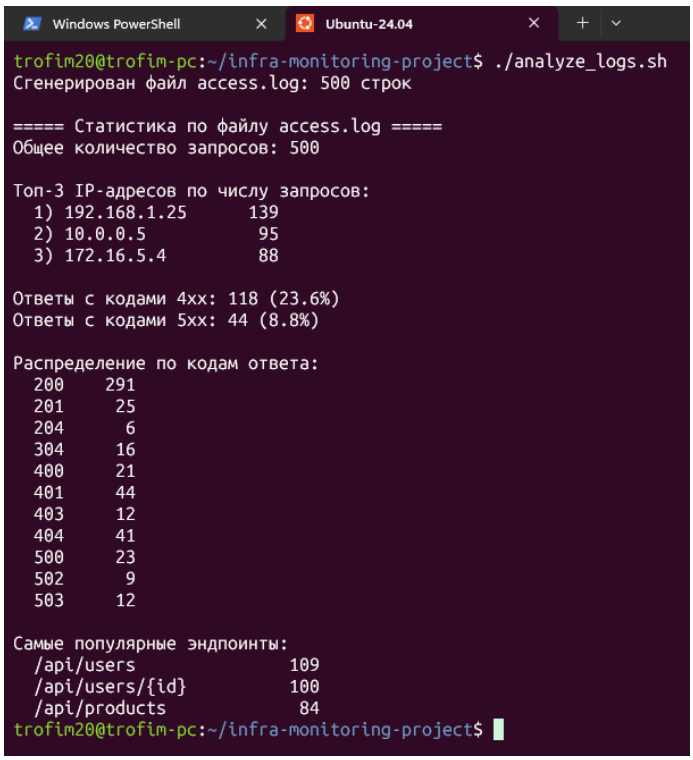


Рисунок 3 – Вывод скрипта analyze_logs.sh в терминале

3.3 Проверка скриптов

Обоим скриптам выданы права на выполнение командой chmod +x. Работа велась в Windows, где у файлов нет бита исполнения, поэтому права дополнительно записаны в индекс Git командой git add --chmod=+x. В репозитории скрипты хранятся с режимом 100755, и после клонирования в Linux их можно сразу запускать.

Синтаксис проверен командами bash -n check_api.sh и bash -n analyze_logs.sh, статический анализатор ShellCheck замечаний не выдал. Для analyze_logs.sh проверены и ошибочные запуски: число строк меньше 300, нечисловое значение -n, неизвестный ключ, лишний аргумент и отсутствующий файл в режиме -a. Во всех случаях скрипт выводит понятное сообщение и возвращает код 1. Основные из этих проверок повторяются в конвейере CI (раздел 5).

4 Docker Compose для стека мониторинга

Стек описан в файле docker-compose.yml в корне проекта (листинг А.3). Он состоит из двух сервисов на готовых образах из Docker Hub (таблица 6).

Таблица 6 – Сервисы стека мониторинга

Сервис	Образ	Сети	Порт	Назначение
api	hashicorp/http-echo:1.0.0	backend	8080, не публикуется на хосте	отвечает текстом «Monitoring Service Active»
proxy	nginx:alpine	frontend, backend	80, опубликован на хосте	обратный прокси, передает запросы в api:8080

Сервис api запускается с аргументами -listen=:8080 и -text=Monitoring Service Active. Версия образа зафиксирована тегом 1.0.0, чтобы стек не поменялся сам при выходе новой версии.

Для связи контейнеров созданы две пользовательские сети с драйвером bridge. Сеть backend объявлена внутренней (internal: true): из нее нет выхода во внешнюю сеть, и порты подключенных только к ней контейнеров нельзя опубликовать на хосте. В этой сети находятся api и proxy. Сеть frontend обычная, через нее proxy публикует порт 80. В итоге снаружи доступен только nginx, а echo-сервис спрятан за ним.

Конфигурация nginx (листинг А.4) подключается в контейнер как шаблон. Образ nginx при запуске подставляет в файлы из /etc/nginx/templates значения переменных окружения, поэтому адрес бэкенда задается прямо в docker-compose.yml переменной API_UPSTREAM=api:8080. Имя api внутри сети разрешает встроенный DNS Docker. Прокси передает все запросы на upstream и добавляет заголовки Host, X-Real-IP, X-Forwarded-For и X-Forwarded-Proto.

Для proxy настроен healthcheck: каждые 15 секунд внутри контейнера выполняется wget http://127.0.0.1/ и проверяется, что в ответе есть текст Monitoring Service Active. Так проверяется вся цепочка от nginx до api, а не только то, что nginx запустился.

Стек запущен командой docker compose up -d. Запрос curl http://localhost:80 вернул текст «Monitoring Service Active», check_api.sh с этим адресом вывел «API доступен», а запрос напрямую на порт 8080 хоста завершился ошибкой соединения, как и должно быть. Состояние контейнеров показано на рисунке 4.

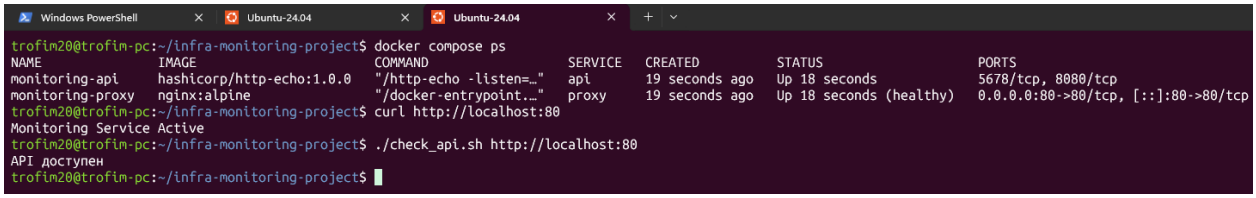


Рисунок 4 – Запущенные контейнеры стека (docker compose ps)

В Docker Compose второй версии команда пишется через пробел: docker compose. Она делает то же самое, что docker-compose из задания.

5 Конвейер CI/CD в GitHub Actions

Конвейер описан в файле .github/workflows/ci.yml (листинг А.5). Он запускается на Pull Request в main, при push в main и вручную (workflow_dispatch). Все задачи выполняются на раннерах ubuntu-latest, их состав приведен в таблице 7.

Таблица 7 – Задачи конвейера CI

Задача	Шаги
lint	1) проверка синтаксиса: bash -n check_api.sh и bash -n analyze_logs.sh; 2) статический анализ ShellCheck; 3) проверка docker-compose.yml командой docker compose config.
test-scripts	1) ./check_api.sh https://httpbin.org/status/200 с ожиданием кода возврата 0; 2) проверка кода 1 на ответе 404, закрытом порту, некорректном URL и запуске без аргументов; 3) генерация журнала, проверка, что в нем не меньше 300 строк, и ошибка при -n 100.
test-compose	1) запуск стека: docker compose up -d --wait; 2) запрос через прокси с проверкой текста и check_api.sh http://localhost:80; 3) проверка, что порт 8080 недоступен с хоста; 4) остановка стека: docker compose down.

Задачи test-scripts и test-compose зависят от lint (needs: lint) и запускаются параллельно только после проверки синтаксиса. Если в скрипте синтаксическая ошибка, тесты не запускаются и не тратят время раннера.

httpbin.org иногда отвечает кодами 502 или 503, поэтому проверка доступного API делает до трех попыток с паузой 5 секунд. Без этого конвейер мог бы падать из-за сбоя внешнего сервиса, а не из-за ошибки в скрипте. Если стек не поднялся, шаг Show logs on failure выводит логи контейнеров, чтобы было видно причину.

На Pull Request #1 все три задачи завершились успешно (рисунок 5). Они же отмечены обязательными проверками в правиле защиты ветки main, поэтому слить в main изменения с ошибкой в скриптах не получится. После

слияния Pull Request конвейер еще раз запустился уже на ветке main (событие push) и тоже прошел успешно.

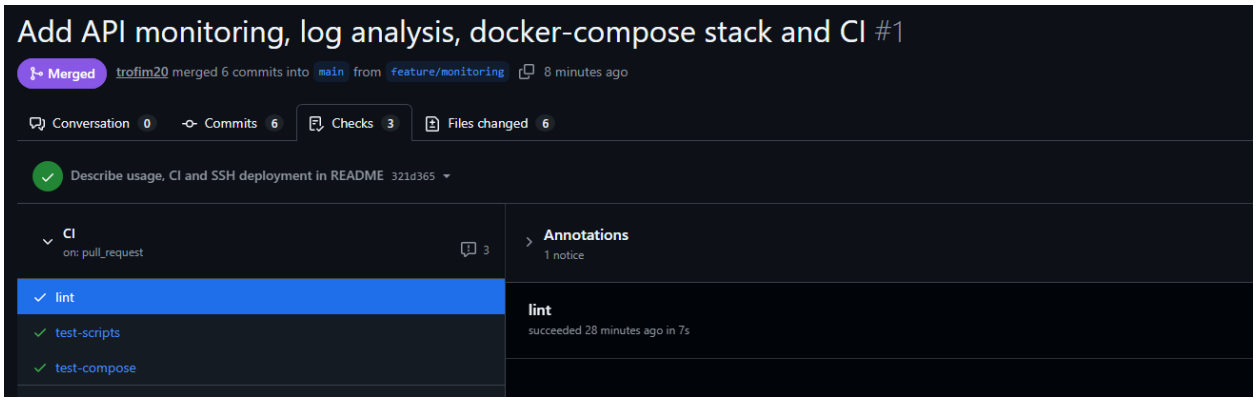


Рисунок 5 – Успешное выполнение конвейера CI в Pull Request #1

6 Передача файлов на удаленный сервер по SSH и SCP

Порядок передачи проекта на удаленный сервер описан в разделе «Деплой на сервер по SSH» файла README.md. В примерах используется сервер 203.0.113.20 (адрес из диапазона для документации) и пользователь deploy.

Сначала на рабочей машине командой ssh-keygen создается пара ключей ed25519. Закрытый ключ остается на машине, при создании его можно защитить парольной фразой. Открытый ключ копируется на сервер командой ssh-copy-id: она один раз спрашивает пароль пользователя и дописывает ключ в ~/.ssh/authorized_keys. Для Windows, где ssh-copy-id нет, в README приведен вариант через ssh и cat. Когда вход по ключу проверен, на сервере отключается вход по паролю и под root (PasswordAuthentication no и PermitRootLogin no в /etc/ssh/sshd_config).

Сервер записывается в ~/.ssh/config под коротким именем monitoring. Проект можно передать двумя способами: упаковать в архив, скопировать его командой scp и распаковать на сервере, или передать архив потоком через tar и ssh без промежуточного файла. Каталог .git и логи в архив не попадают. После передачи стек запускается на сервере командой docker compose up -d и проверяется скриптом check_api.sh. Команды приведены в листинге 4.

Листинг 4 – Команды для деплоя проекта на сервер

```
# 1. Ключ и вход по ключу
ssh-keygen -t ed25519 -C "infra-monitoring deploy" -f ~/.ssh/infra_deploy
ssh-copy-id -i ~/.ssh/infra_deploy.pub deploy@203.0.113.20
ssh -i ~/.ssh/infra_deploy deploy@203.0.113.20 'whoami && hostname'

# 2. Архив и передача через scp
tar -czf infra-monitoring.tar.gz --exclude=.git --exclude='*.log' .
scp infra-monitoring.tar.gz monitoring:/tmp/
ssh monitoring 'tar -xzf /tmp/infra-monitoring.tar.gz -C /opt/infra-monitoring'

# 3. То же через tar + ssh, без временного файла
tar -czf - --exclude=.git --exclude='*.log' . \
    | ssh monitoring 'tar -xzf - -C /opt/infra-monitoring'

# 4. Запуск стека и проверка
ssh monitoring 'cd /opt/infra-monitoring && docker compose up -d'
ssh monitoring '/opt/infra-monitoring/check_api.sh http://localhost:80'
```

ЗАКЛЮЧЕНИЕ

Все шесть этапов задания выполнены. В публичном репозитории infra-monitoring-project ветка main защищена правилами GitHub, разработка велась в ветке feature/monitoring и после прохождения CI слита в main через Pull Request с описанием изменений.

В Wireshark снят трафик запросов к httpbin.org. По дампу определены участники обмена, методы GET, POST и DELETE и коды ответов 200 и 404. На примере HTTPS-запроса видно, что при шифровании из трафика можно узнать только адрес и имя сервера.

Скрипт check_api.sh проверяет доступность API и возвращает 0 или 1, analyze_logs.sh генерирует журнал из 500 записей и считает по нему статистику с помощью grep, awk, sed, sort, uniq и регулярных выражений. Стек Docker Compose из http-echo и nginx отвечает через прокси текстом «Monitoring Service Active», при этом echo-сервис находится во внутренней сети и снаружи напрямую недоступен. Конвейер GitHub Actions проверяет синтаксис и работу скриптов и запуск стека и проходит успешно. В README описаны создание SSH-ключа, копирование ключа на сервер и передача проекта через scp и tar + ssh.

ПРИЛОЖЕНИЕ А. Листинги скриптов и конфигурации

Листинг А.1 – Скрипт check_api.sh

```
#!/usr/bin/env bash
#
# check_api.sh - checks API availability by HTTP status code.
#
# Usage:  ./check_api.sh [-t SECONDS] [-v] URL
# Exit:   0 if the server answered 200, 1 in any other case.

set -o nounset
set -o pipefail

readonly EXPECTED_STATUS=200

timeout=10
verbose=0

usage() {
    cat <<EOF
Использование: ${0##*/} [-t СЕКУНДЫ] [-v] URL

Отправляет HTTP-запрос на URL и проверяет,
что сервер ответил кодом ${EXPECTED_STATUS}.

Ключи:
  -t СЕКУНДЫ  таймаут запроса, по умолчанию ${timeout}
  -v          дополнительно вывести URL и время ответа
  -h          показать эту справку

Коды возврата: 0 - API доступен, 1 - API недоступен или ошибка запуска.
EOF
}

error() {
    printf 'Ошибка: %s\n' "$*" >&2
}

# Short explanation for the most common curl failures
curl_reason() {
    case $1 in
        6)  echo "не удалось определить адрес хоста" ;;
        7)  echo "не удалось подключиться к серверу" ;;
        28) echo "превышено время ожидания ответа" ;;
        35|60) echo "ошибка TLS-соединения" ;;
        *)  echo "curl завершился с кодом $1" ;;
    esac
}

while getopts ':t:vh' opt; do
    case $opt in
        t) timeout=$OPTARG ;;
        v) verbose=1 ;;
        h) usage; exit 0 ;;
        :) error "для ключа -$OPTARG нужно значение"; exit 1 ;;
        *) error "неизвестный ключ -$OPTARG"; usage >&2; exit 1 ;;
    esac
done
shift $((OPTIND - 1))

if (( $# != 1 )); then
    error "нужно передать один аргумент: URL API"
    usage >&2
    exit 1
```

```
fi

url=$1

if ! [[ $timeout =~ ^[1-9][0-9]*$ ]]; then
    error "таймаут должен быть целым положительным числом: $timeout"
    exit 1
fi

# http(s)://host[:port][/path], host is letters, digits, dots and hyphens
scheme_re='https?://'
host_re='[A-Za-z0-9]([A-Za-z0-9.-]*[A-Za-z0-9])?'
port_re='(:[0-9]{1,5})?'
path_re='(/[^[:space:]]*)?'
url_regex="^${scheme_re}${host_re}${port_re}${path_re}\$"

if ! [[ $url =~ $url_regex ]]; then
    error "некорректный URL: $url (нужен http:// или https://)"
    exit 1
fi

if ! command -v curl >/dev/null 2>&1; then
    error "не найден curl"
    exit 1
fi

# Response body is not needed, only the status code and the total time.
# If the request fails, curl still prints 000 as the status code.
curl_exit=0
response=$(curl --silent --output /dev/null --max-time "$timeout" \
    --write-out '%{http_code} %{time_total}' "$url") || curl_exit=$?
read -r status time_total <<< "$response"

if [[ $status == "$EXPECTED_STATUS" ]]; then
    echo "API доступен"
    rc=0
else
    echo "API недоступен, код ответа: $status"
    rc=1
fi

if (( curl_exit != 0 )); then
    echo "Причина: $(curl_reason "$curl_exit")" >&2
fi

if (( verbose )); then
    echo "URL: $url"
    echo "Время ответа: ${time_total} с"
fi

exit "$rc"
```

Листинг А.2 – Скрипт analyze_logs.sh

```
#!/usr/bin/env bash
#
# analyze_logs.sh - generates a test access.log and prints statistics for it.
#
# Usage:  ./analyze_logs.sh [-n LINES] [-f FILE] [-s SEED] [-a] [-h]

set -euo pipefail

readonly MIN_LINES=300
```

```
lines=500
log_file=access.log
seed=2026
analyze_only=0

# IPv4 address: four groups of 1-3 digits separated by dots.
# Groups above 255 are filtered out later with awk.
readonly IP_REGEX='([0-9]{1,3}\.){3}[0-9]{1,3}'

# Full log line: IP - - [date] "METHOD /path HTTP/x.x" CODE
readonly DATE_REGEX='\[[^]]+\]'
readonly REQ_REGEX='"[A-Z]+ [^ "]+ HTTP/[0-9.]+"'
readonly LINE_REGEX="^${IP_REGEX} - - ${DATE_REGEX} ${REQ_REGEX} [0-9]{3}\$"

# Test data in "value:weight" form, a bigger weight means the value
# shows up in the log more often
readonly CLIENT_IPS=(
    "192.168.1.25:9" "10.0.0.5:7" "172.16.5.4:6" "192.168.1.10:4"
    "10.0.0.12:3" "192.168.1.33:2" "203.0.113.45:2" "198.51.100.7:1"
)
readonly REQUESTS=(
    "GET /api/users:10" "GET /api/users/{id}:8" "GET /api/products:8"
    "GET /api/products/{id}:6" "POST /api/login:7" "POST /api/orders:4"
    "PUT /api/users/{id}:2" "DELETE /api/orders/{id}:2" "GET /health:3"
)
readonly STATUSES=(
    "200:62" "304:4" "400:4" "401:3" "403:2" "404:9"
    "500:4" "502:2" "503:3"
)

usage() {
    cat <<EOF
Использование: ${0##*/} [-n СТРОК] [-f ФАЙЛ] [-s ЧИСЛО] [-a] [-h]

Генерирует тестовый лог веб-сервера и выводит статистику по нему:
число запросов, топ-3 IP-адресов, число ответов с кодами 4xx и 5xx.

Ключи:
  -n СТРОК  число строк в логе: от ${MIN_LINES}, по умолчанию ${lines}
  -f ФАЙЛ   файл лога, по умолчанию ${log_file}
  -s ЧИСЛО  зерно генератора случайных чисел, по умолчанию ${seed}
  -a        не генерировать лог, только проанализировать существующий файл
  -h        показать эту справку
EOF
}

fail() {
    printf 'Ошибка: %s\n' "$*" >&2
    exit 1
}

# Linear congruential generator. Bash's own RANDOM may differ between
# versions, this one gives the same log on any machine for the same seed.
rand_state=0
next_random() {
    rand_state=$(( (rand_state * 1103515245 + 12345) % 2147483648 ))
    REPLY=$(( rand_state / 65536 % $1 ))
}

# Expands "value:weight" pairs into an array where each value
# is repeated weight times. Usage: make_pool ARRAY_NAME PAIR...
make_pool() {
    local -n pool=$1
    shift
    local pair value weight
    pool=()
    for pair in "$@"; do
```

```
        value=${pair%:*}
        weight=${pair##*:}
        while (( weight-- > 0 )); do
            pool+=("$value")
        done
    done
}

generate_log() {
    local file=$1 count=$2
    local -a ip_pool request_pool status_pool
    local i ip request status id
    local placeholder='{id}'
    local sec=$(( 10 * 3600 + 15 * 60 + 32 ))   # first record at 10:15:32

    make_pool ip_pool "${CLIENT_IPS[@]}"
    make_pool request_pool "${REQUESTS[@]}"
    make_pool status_pool "${STATUSES[@]}"
    rand_state=$(( 10#$seed ))

    for (( i = 0; i < count; i++ )); do
        next_random "${#ip_pool[@]}"
        ip=${ip_pool[REPLY]}

        next_random "${#request_pool[@]}"
        request=${request_pool[REPLY]}
        if [[ $request == *"$placeholder"* ]]; then
            next_random 500
            id=$(( REPLY + 1 ))
            request=${request/"$placeholder"/$id}
        fi

        # Login fails with 401 in about a third of attempts,
        # successful POST and DELETE get 201 and 204
        if [[ $request == "POST /api/login" ]]; then
            next_random 3
            status=$(( REPLY == 0 ? 401 : 200 ))
        else
            next_random "${#status_pool[@]}"
            status=${status_pool[REPLY]}
            if [[ $status == 200 && $request == POST* ]]; then
                status=201
            elif [[ $status == 200 && $request == DELETE* ]]; then
                status=204
            fi
        fi

        printf '%s - - [15/Aug/2026:%02d:%02d:%02d] "%s HTTP/1.1" %s\n' \
            "$ip" $(( sec / 3600 % 24 )) $(( sec / 60 % 60 )) $(( sec % 60 )) \
            "$request" "$status"

        next_random 4
        sec=$(( sec + REPLY ))
    done > "$file"
}

analyze_log() {
    local file=$1
    local total_lines requests bad_lines errors_4xx errors_5xx

    total_lines=$(wc -l < "$file")
    requests=$(grep -cE "$LINE_REGEX" "$file" || true)
    bad_lines=$(( total_lines - requests ))

    echo "===== Статистика по файлу ${file} ====="
    echo "Общее количество запросов: ${requests}"
    if (( bad_lines > 0 )); then
```

```
        echo "Пропущено строк в неверном формате: ${bad_lines}"
    fi
    echo

    echo "Топ-3 IP-адресов по числу запросов:"
    grep -oE "^${IP_REGEX}" "$file" |
        awk -F. '$1 <= 255 && $2 <= 255 && $3 <= 255 && $4 <= 255' |
        sort | uniq -c | sort -k1,1nr -k2,2 |
        awk 'NR <= 3 { printf "  %d) %-15s %5d\n", NR, $2, $1 }'
    echo

    # Status code is the last field of the line
    read -r errors_4xx errors_5xx < <(
        awk '$NF ~ /^4[0-9][0-9]$/ { c4++ }
             $NF ~ /^5[0-9][0-9]$/ { c5++ }
             END { print c4 + 0, c5 + 0 }' "$file"
    )
    awk -v c4="$errors_4xx" -v c5="$errors_5xx" -v total="$requests" 'BEGIN {
        if (total == 0) total = 1
        printf "Ответы с кодами 4xx: %d (%.1f%%)\n", c4, c4 * 100 / total
        printf "Ответы с кодами 5xx: %d (%.1f%%)\n", c5, c5 * 100 / total
    }'
    echo

    echo "Распределение по кодам ответа:"
    awk '$NF ~ /^[1-5][0-9][0-9]$/ { print $NF }' "$file" |
        sort | uniq -c |
        awk '{ printf "  %s  %5d\n", $2, $1 }'
    echo

    # Numeric ids are replaced with {id}, so /api/users/7 and
    # /api/users/42 are counted as one endpoint
    echo "Самые популярные эндпоинты:"
    sed -nE 's/^[^"]*"[A-Z]+ ([^ "?]+)[^"]*".*$/\1/p' "$file" |
        sed -E 's#/[0-9]+$#/{id}#' |
        sort | uniq -c | sort -k1,1nr -k2,2 |
        awk 'NR <= 3 { printf "  %-22s %5d\n", $2, $1 }'
}

while getopts ':n:f:s:ah' opt; do
    case $opt in
        n) lines=$OPTARG ;;
        f) log_file=$OPTARG ;;
        s) seed=$OPTARG ;;
        a) analyze_only=1 ;;
        h) usage; exit 0 ;;
        :) fail "для ключа -$OPTARG нужно значение" ;;
        *) echo "Ошибка: неизвестный ключ -$OPTARG" >&2; usage >&2; exit 1 ;;
    esac
done
shift $((OPTIND - 1))

(( $# == 0 )) || fail "лишние аргументы: $*"

for tool in grep awk sed sort uniq wc; do
    command -v "$tool" >/dev/null 2>&1 || fail "не найдена утилита $tool"
done

if (( ! analyze_only )); then
    [[ $lines =~ ^[0-9]+$ ]] || fail "число строк должно быть целым: $lines"
    (( 10#$lines >= MIN_LINES )) || fail "нужно не меньше $MIN_LINES строк"
    [[ $seed =~ ^[0-9]+$ ]] || fail "зерно -s должно быть целым: $seed"

    generate_log "$log_file" "$(( 10#$lines ))"
    echo "Сгенерирован файл ${log_file}: $(wc -l < "$log_file") строк"
    echo
fi
```

```
[[ -f $log_file ]] || fail "файл $log_file не найден"
[[ -s $log_file ]] || fail "файл $log_file пустой"

analyze_log "$log_file"
```

Листинг А.3 – Файл docker-compose.yml

```
# Monitoring stack: echo API behind an nginx reverse proxy.
#
#   client -> proxy (nginx, :80) -> api (http-echo, :8080)
#
# api is connected only to the internal "backend" network, so it is not
# reachable from the host directly. proxy is connected to both networks
# and is the only service that publishes a port.

services:
  api:
    image: hashicorp/http-echo:1.0.0
    container_name: monitoring-api
    command:
      - "-listen=:8080"
      - "-text=Monitoring Service Active"
    expose:
      - "8080"
    networks:
      - backend
    restart: unless-stopped

  proxy:
    image: nginx:alpine
    container_name: monitoring-proxy
    ports:
      - "80:80"
    environment:
      # substituted into nginx/templates/default.conf.template at startup
      API_UPSTREAM: api:8080
    volumes:
      - ./nginx/templates:/etc/nginx/templates:ro
    depends_on:
      - api
    networks:
      - frontend
      - backend
    healthcheck:
      # checks the whole chain: nginx is up and api answers through it
      test:
        - CMD-SHELL
        - wget -qO- http://127.0.0.1/ | grep -q 'Monitoring Service Active'
      interval: 15s
      timeout: 5s
      retries: 3
      start_period: 5s
    restart: unless-stopped

networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
    internal: true
```

Листинг А.4 – Шаблон конфигурации nginx/templates/default.conf.template

```
# Reverse proxy for the echo API.
# ${API_UPSTREAM} is replaced by the nginx image (envsubst) when the
# container starts, the name "api" is resolved by Docker's internal DNS.

upstream echo_api {
    server ${API_UPSTREAM};
}

server {
    listen      80;
    server_name _;

    location / {
        proxy_pass         http://echo_api;
        proxy_http_version 1.1;

        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_connect_timeout 5s;
        proxy_read_timeout    10s;
    }
}
```

Листинг А.5 – Конвейер CI .github/workflows/ci.yml

```
name: CI

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  lint:
    name: lint
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v5

      - name: Check bash syntax
        run: |
          bash -n check_api.sh
          bash -n analyze_logs.sh

      - name: Run ShellCheck
        run: shellcheck check_api.sh analyze_logs.sh

      - name: Validate docker-compose.yml
        run: docker compose config --quiet

  test-scripts:
    name: test-scripts
    runs-on: ubuntu-latest
    needs: lint
    steps:
      - name: Checkout
```

```
      uses: actions/checkout@v5

    - name: check_api.sh returns 0 for a working API
      run: |
        # httpbin.org sometimes answers 502/503, so try a few times
        for attempt in 1 2 3; do
          if ./check_api.sh https://httpbin.org/status/200; then
            exit 0
          fi
          echo "Attempt $attempt failed, retrying in 5 seconds"
          sleep 5
        done
        exit 1

    - name: check_api.sh returns 1 on errors
      run: |
        must_fail() {
          if ./check_api.sh "$@"; then
            echo "Expected exit code 1 for: $*"
            exit 1
          fi
        }
        must_fail https://httpbin.org/status/404
        must_fail http://127.0.0.1:9
        must_fail not-a-url
        must_fail

    - name: analyze_logs.sh generates and analyzes the log
      run: |
        ./analyze_logs.sh | tee report.txt
        test "$(wc -l < access.log)" -ge 300
        grep -q "Общее количество запросов: 500" report.txt
        if ./analyze_logs.sh -n 100; then
          echo "Expected an error for less than 300 lines"
          exit 1
        fi

  test-compose:
    name: test-compose
    runs-on: ubuntu-latest
    needs: lint
    steps:
      - name: Checkout
        uses: actions/checkout@v5

      - name: Start the stack
        run: docker compose up -d --wait

      - name: Request through the proxy
        run: |
          docker compose ps
          curl -s http://localhost:80 | tee response.txt
          grep -q "Monitoring Service Active" response.txt
          ./check_api.sh http://localhost:80

      - name: api is not published on the host
        run: |
          if curl -s --max-time 3 http://localhost:8080; then
            echo "Port 8080 must not be reachable from the host"
            exit 1
          fi

      - name: Show logs on failure
        if: failure()
        run: docker compose logs

      - name: Stop the stack
```

```
        if: always()
        run: docker compose down
```